ΤΕΧΝΟΛΟΓΙΑ ΛΟΓΙΣΜΙΚΟΥ

Το εργαστήριο

Από την ιδέα και τον σχεδιασμό
σε κώδικα που λειτουργεί.

Δρ. Ιωάννης Στύλιος

Αξιολόγηση

Εργασία - Εξέταση

Δομή Βαθμολογίας: Το μάθημα περιλαμβάνει δύο προόδους, οι οποίες είναι απαλλακτικές για όσους πετύχουν προβιβάσιμο βαθμό. Η κατανομή του βαθμού διαμορφώνεται ως εξής:

- Πρώτη πρόοδος: 50%
- Δεύτερη πρόοδος: 20% ή 50%
- Εργασία: 30%

Εναλλακτική Επιλογή:

- Όσοι συμμετέχουν στις προόδους αλλά δεν υποβάλουν εργασία ο τελικός βαθμός προκύπτει από τις 2 προόδους.
- Όσοι δώσουν μόνο μια πρόοδο το υπόλοιπο 50% ή 20% (αν δώσουν εργασία) θα δοθεί στην τελική εξέταση.
- Όσοι δεν συμμετάσχουν στις προόδους και στην εργασία, η τελική βαθμολογία θα προκύψει αποκλειστικά από την τελική εξέταση.

Εξέταση Εργαστηρίου (Προαιρετική):

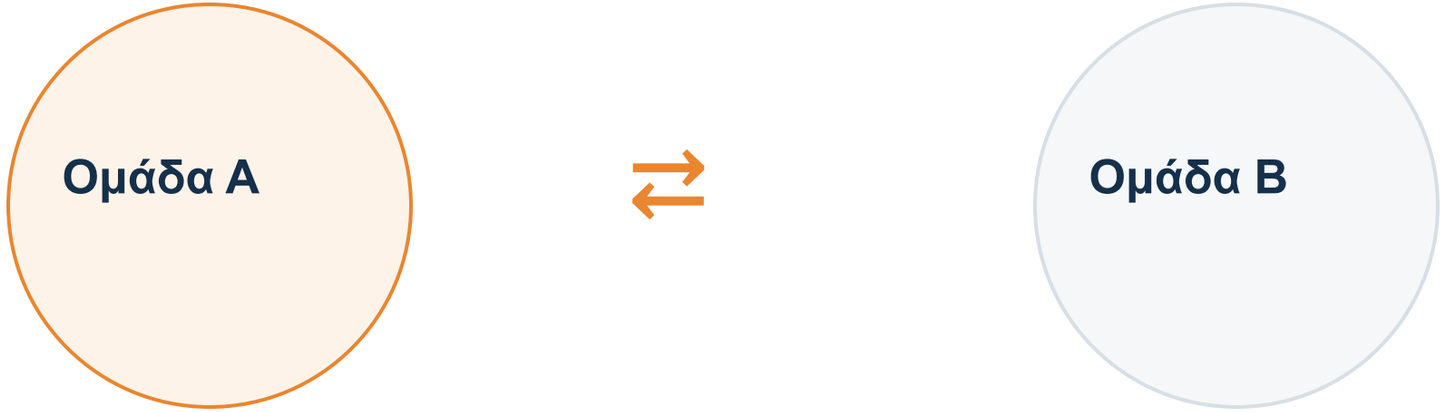
- Η εξέταση του εργαστηρίου είναι προαιρετική και μπορεί μόνο να ενισχύσει τον τελικό βαθμό σας.

- Οι εξετάσεις: με ανοιχτές σημειώσεις μαθήματος.

- Κατοχύρωση:
  – Ο βαθμός από τις εργασίες κατοχυρώνεται μέχρι και το Σεπτέμβριο.
  – Τίποτε δεν κατοχυρώνεται για επόμενη χρονιά!

Πώς οργανώνονται τα τμήματα

Δύο ομάδες: Α και Β



Τα τμήματα γίνονται εναλλάξ: κάθε ομάδα παρακολουθεί κάθε δεύτερη εβδομάδα.

Συμμετοχή και bonus βαθμού

Το εργαστήριο είναι προαιρετικό.

Η προαιρετική αξιολόγηση του εργαστηρίου μπορεί να προσθέσει

έως +1 μονάδα

στον τελικό βαθμό του μαθήματος.

Σχεδίαση λογισμικού

Εργαλείο σχεδίασης UML: diagrams.net (draw.io)

Κύκλος ζωής και μοντέλα

Εφαρμογή και σύγκριση
μοντέλων ανάπτυξης

Απαιτήσεις

Ανάλυση και τεκμηρίωση
αναγκών των χρηστών

Διαγράμματα UML

Περιπτώσεις χρήσης
και διαγράμματα ακολουθίας
Εργαλείο: diagrams.net (draw.io)

Σχεδίαση κλάσεων

Διαγράμματα κλάσεων
και αντικειμένων

Kotlin, υλοποίηση και έλεγχος

Εργαλείο προγραμματισμού: IntelliJ IDEA **https://www.jetbrains.com/idea/**

Βασικά στοιχεία Kotlin

Μεταβλητές, τύποι δεδομένων
και τελεστές
Εργαλείο: IntelliJ IDEA, Kotlin Playground

Λογική προγράμματος

Δομές ελέγχου και επανάληψης,
πίνακες και λίστες

Συναρτήσεις και κλάσεις

Συναρτήσεις, κλάσεις
και αντικείμενα

Από την UML στον κώδικα

Διεπαφές, αφηρημένες κλάσεις
και δοκιμές

Kotlin

Ε1. ΕΙΣΑΓΩΓΗ

ΤΙ ΕΙΝΑΙ Η KOTLIN;

Βασικά Στοιχεία

✨ Δημιουργήθηκε από την JetBrains το 2011

🚀 Πρώτη σταθερή έκδοση (v1.0) το 2016

📱 Επίσημη γλώσσα για Android (Google, 2017)

☕ Εκτελείται σε JVM (Java Virtual Machine)

📖 Ανοιχτού κώδικα και δωρεάν

Δημιουργός: Andrey Breslav (Lead Designer)

Εταιρεία: JetBrains - οι δημιουργοί του IntelliJ IDEA. η Kotlin αναπτύχθηκε από μια διεθνή εταιρεία λογισμικού, με ρωσικές ιδρυτικές ρίζες και σημερινή έδρα την Ολλανδία.

Ονομασία: Από το νησί Kotlin κοντά στην Αγία Πετρούπολη

Σήμερα: Χρησιμοποιείται από Google, Netflix, Amazon, Uber, Pinterest

ΓΙΑΤΙ KOTLIN;

🎯 ΣΥΝΟΠΤΙΚΉ & ΚΑΘΑΡΉ

• Γράφετε λιγότερο κώδικα για το ίδιο αποτέλεσμα

• Μέχρι 40% λιγότερες γραμμές από Java

• Λιγότερος περιττός κώδικας σημαίνει λιγότερα bugs

🔄 ΣΥΜΒΑΤΗ ΜΕ JAVA

• 100% διαλειτουργικότητα

• Χρήση όλων των Java βιβλιοθηκών

🛡️ ΑΣΦΑΛΗΣ

• Προστασία από NullPointerException (το #1 πρόβλημα στη Java)

• Εντοπίζει σφάλματα null πριν καν τρέξει το πρόγραμμα

• Λιγότερα crashes, πιο σταθερές εφαρμογές

🧰 ΕΥΚΟΛΗ ΣΤΗ ΧΡΗΣΗ

• Άριστη υποστήριξη σε IDE (αυτόματη συμπλήρωση, προτάσεις)

• Υποστήριξη στο IntelliJ IDEA και Android Studio

• Βοηθάει στη δημιουργία σωστού κώδικα πιο γρήγορα

ΠΕΡΙΒΑΛΛΟΝ ΑΝΑΠΤΥΞΗΣ

ΕΠΙΛΟΓΕΣ ΕΡΓΑΛΕΙΩΝ

💻 IntelliJ IDEA

Το καλύτερο IDE για Kotlin

Community Edition: Δωρεάν

🤖 Android Studio

Για mobile εφαρμογές Android

🌐 Kotlin Playground

Online: play.kotlinlang.org

Άμεση εκτέλεση χωρίς εγκατάσταση!

ΤΟ ΠΡΩΤΟ ΠΡΟΓΡΑΜΜΑ!

Hello World!

```
fun main() {
    println("Hello, Kotlin!")
}
```

Αποτέλεσμα:

```
Hello, Kotlin!
```

🔧

fun

Λέξη-κλειδί για συναρτήσεις

🚀

main()

Σημείο εκκίνησης

📢

println()

Εκτύπωση με νέα γραμμή

✅ Όχι υποχρεωτικά semicolons

ΔΟΜΗ ΠΡΟΓΡΑΜΜΑΤΟΣ

Η συνάρτηση main()

Κάθε πρόγραμμα ξεκινάει από εδώ

Σύγχρονη Μορφή:

```
fun main() {
    println("Hello!")
}
```

Η απλούστερη μορφή - χρησιμοποιείται στα περισσότερα προγράμματα που δεν χρειάζονται εισόδους από τον χρήστη κατά την εκκίνηση.

Σχόλια (Comments)

Μιας γραμμής:

```
// Αυτό είναι σχόλιο
```

Πολλών γραμμών:

```
/*
 * Σχόλιο
 * πολλών γραμμών
 */
```

Output - Εμφάνιση στην οθόνη

println()
Εκτυπώνει και προσθέτει νέα γραμμή

```
println("Γεια!")
println("Καλημέρα!")
```

```
Γεια!
Καλημέρα!
```

print()
Εκτυπώνει χωρίς νέα γραμμή

```
print("Γεια ")
print("σου")
```

```
Γεια σου
```

ΜΕΤΑΒΛΗΤΕΣ: VAL VS VAR

val (Αμετάβλητη)

```
val name = "Alice"

// Σφάλμα!
// name = "Bob"
```

✅ Δεν μπορεί να αλλάξει

✅ Προτιμάται για ασφάλεια

var (Μεταβλητή)

```
var score = 0
var level = 1

// OK!
score = 100
```

✅ Μπορεί να αλλάξει

⚠️ Χρήση όταν χρειάζεται

ΠΡΟΤΙΜΗΣΤΕ VAL ΩΣ ΠΡΟΕΠΙΛΟΓΗ!

ΓΙΑΤΙ ΠΡΟΤΙΜΑΜΕ VAL;

🔒 Πιο ασφαλής - αποφυγή ακούσιων αλλαγών

ΤΥΠΟΙ ΔΕΔΟΜΕΝΩΝ

Αριθμητικοί

```
val int: Int = 42
val long: Long = 42L
val double: Double = 3.14
val float: Float = 3.14f
```

Άλλοι Τύποι

```
val char: Char = 'A'
val text: String = "Hello"
val flag: Boolean = true
```

Η Kotlin καταλαβαίνει τον τύπο αυτόματα (Type Inference)

```
val number = 42        // Int
val price = 19.99      // Double
val message = "Kotlin" // String
```

```
fun main() {
    val number = 42
    val price = 19.99
    val message = "Kotlin"

    println(number::class.simpleName)   // Int
    println(price::class.simpleName)    // Double
    println(message::class.simpleName)  // String
}
```

ΒΑΣΙΚΟΙ ΤΕΛΕΣΤΕΣ - ΑΡΙΘΜΗΤΙΚΟΙ

Βασικές Πράξεις

```
val a = 10
val b = 3

val sum = a + b      // 13
val diff = a - b     // 7
val prod = a * b     // 30
val quot = a / b     // 3
val rem = a % b      // 1
```

Εξήγηση:

+ Πρόσθεση

- Αφαίρεση

* Πολλαπλασιασμός

/ Διαίρεση

% Υπόλοιπο

Σύντομες Εκχωρήσεις

Μακριά:

```
x = x + 3
x = x - 2
```

Σύντομη:

```
x += 3
x -= 2
```

ΒΑΣΙΚΟΙ ΤΕΛΕΣΤΕΣ - ΣΥΓΚΡΙΣΗΣ & ΛΟΓΙΚΟΙ

Τελεστές Σύγκρισης

Επιστρέφουν true ή false

```
val a = 5
val b = 10

a == b  // false (ίσο)
a != b  // true (διάφορο)
a < b   // true (μικρότερο)
a > b   // false (μεγαλύτερο)
```

== Ίσο

!= Διάφορο

< Μικρότερο

> Μεγαλύτερο

<= Μικρότερο ή ίσο

>= Μεγαλύτερο ή ίσο

Λογικοί Τελεστές

Έστω age = 20 και hasLicense = true.

AND (&&) age >= 18 && hasLicense → true
Το άτομο είναι τουλάχιστον 18 ετών **και** έχει δίπλωμα.

OR (||) age < 18 || age > 65 → false
Το άτομο δεν είναι ούτε κάτω των 18 ούτε άνω των 65 ετών.

NOT (!) !(age > 65) → true
Η πρόταση «είναι άνω των 65» είναι false, οπότε η άρνησή της είναι true.

NULL SAFETY

ΤΟ ΠΡΟΒΛΗΜΑ ΣΤΗ JAVA

NullPointerException

Σφάλμα που συμβαίνει όταν προσπαθούμε να χρησιμοποιήσουμε μια μεταβλητή που είναι κενή (null). Ο #1 λόγος crashes στη Java.

```
String name = null;
name.length(); // CRASH!
```

Η ΛΥΣΗ ΣΤΗΝ KOTLIN

Η Kotlin προστατεύει από null! Διακρίνει μεταβλητές που μπορούν να είναι null από αυτές που δεν μπορούν.

Non-Nullable

```
var name: String = "Alice"
// name = null // Σφάλμα!
```

Δεν επιτρέπεται null

Nullable (με ?)

```
var name: String? = "Alice"
name = null // OK!
```

Το ? επιτρέπει null

NULL SAFETY - ELVIS OPERATOR

Elvis Operator: ?:

Τι κάνει;

Παρέχει εναλλακτική τιμή όταν κάτι είναι null. Λειτουργεί σαν: "Αν είναι null, τότε χρησιμοποίησε αυτή την τιμή".

```
val name: String? = null
val display = name ?: "Επισκέπτης"
println(display)
```

```
Επισκέπτης
```

Επειδή το name είναι null, χρησιμοποιείται η εναλλακτική τιμή "Επισκέπτης".

💡 **Πώς να το θυμάστε:**

A ?: B

"Θέλω το A, αλλά αν είναι null τότε δώσε μου το B ως εφεδρική λύση"

Παράδειγμα: name ?: "Επισκέπτης" = "Θέλω το όνομα, αλλά αν δεν υπάρχει, χρησιμοποίησε 'Επισκέπτης'"

STRING TEMPLATES

Παλιός Τρόπος (Java)

Συνένωση με +

```
String onoma = "Μαρία";
int age = 25;
String msg = "Γεια σου" +
    onoma +
    " είσαι " +
    age;
```

```
Γεια σου Μαρία είσαι 25
```

❌ Δύσκολο, αργό

Νέος Τρόπος (Kotlin)

Με $ σύμβολο

```
val name = "Μαρία"
val age = 25
val msg = "Γεια σου $name είσαι $age"
println(msg)
```

```
Γεια σου Μαρία είσαι 25
```

✅ Εύκολο, γρήγορο

Με Εκφράσεις

Για πιο σύνθετα, χρησιμοποιούμε αγκύλες

```
val price = 100
println("Με ΦΠΑ: ${price * 1.24}€")
```

```
Με ΦΠΑ: 124.0€
```

INPUT/OUTPUT - ΕΙΣΟΔΟΣ/ΕΞΟΔΟΣ

Input - Διάβασμα από χρήστη

readln()
Ζητάει από τον χρήστη να πληκτρολογήσει κείμενο και πατήσει Enter.
Επιστρέφει String.

Παράδειγμα:

```
fun main() {
    print("Πώς σε λένε; ")
    val name = readln().ifBlank { "Επισκέπτης" }
    println("Γεια σου, $name!")
}
```

```
Πώς σε λένε; Μαρία
Γεια σου, Μαρία!
```

Ανάγνωση αριθμού

Η readln() δίνει String.
Η toInt() το μετατρέπει σε Int.

```
fun main() {
  print("Ηλικία: ")
  val age = readln().toInt()
  println("Είσαι $age ετών")
}
```

```
Ηλικία: 20
Είσαι 20 ετών
```

ΑΣΚΗΣΗ ΕΡΓΑΣΤΗΡΙΟΥ

Χαιρετισμός και ηλικία

Εκφώνηση

Σε πρόγραμμα που τρέχει σε τερματικό, ζητήστε από τον χρήστη το όνομα και την ηλικία του. Θεωρήστε ότι η ηλικία δίνεται ως ακέραιος.

Υπολογίστε πόσων ετών θα είναι σε έναν χρόνο. Εμφανίστε έναν χαιρετισμό και τη νέα ηλικία χρησιμοποιώντας το $ στα μηνύματα.

.

Παράδειγμα εισόδου

Όνομα: Μαρία
Ηλικία: 20

Συναρτήσεις:
readln()
readln().toInt()
println()

Αναμενόμενο: «Γεια σου, Μαρία!» και «Σε έναν χρόνο θα είσαι 21 ετών».

ΛΥΣΗ ΑΣΚΗΣΗΣ

Χαιρετισμός και ηλικία

```
fun main() {
    print("Όνομα: ")
    val name = readln()
    print("Ηλικία: ")
    val age = readln().toInt()
    val ageNextYear = age + 1

    println("Γεια σου, $name!")
    println("Σε έναν χρόνο θα είσαι $ageNextYear ετών.")
}
```

Αποτέλεσμα: Γεια σου, Μαρία!
Σε έναν χρόνο θα είσαι 21 ετών.

ΕΠΟΜΕΝΟ ΜΑΘΗΜΑ

Επόμενο Μάθημα

Εβδομάδα 2

📊 Δομές ελέγχου (if, when)

🔁 Δομές επανάληψης (for, while)

🎯 Πρακτικές εφαρμογές



Εισαγωγή στην Τεχνολογία Λογισμικού.

Thanks for Watching

i.stylios@uoi.gr